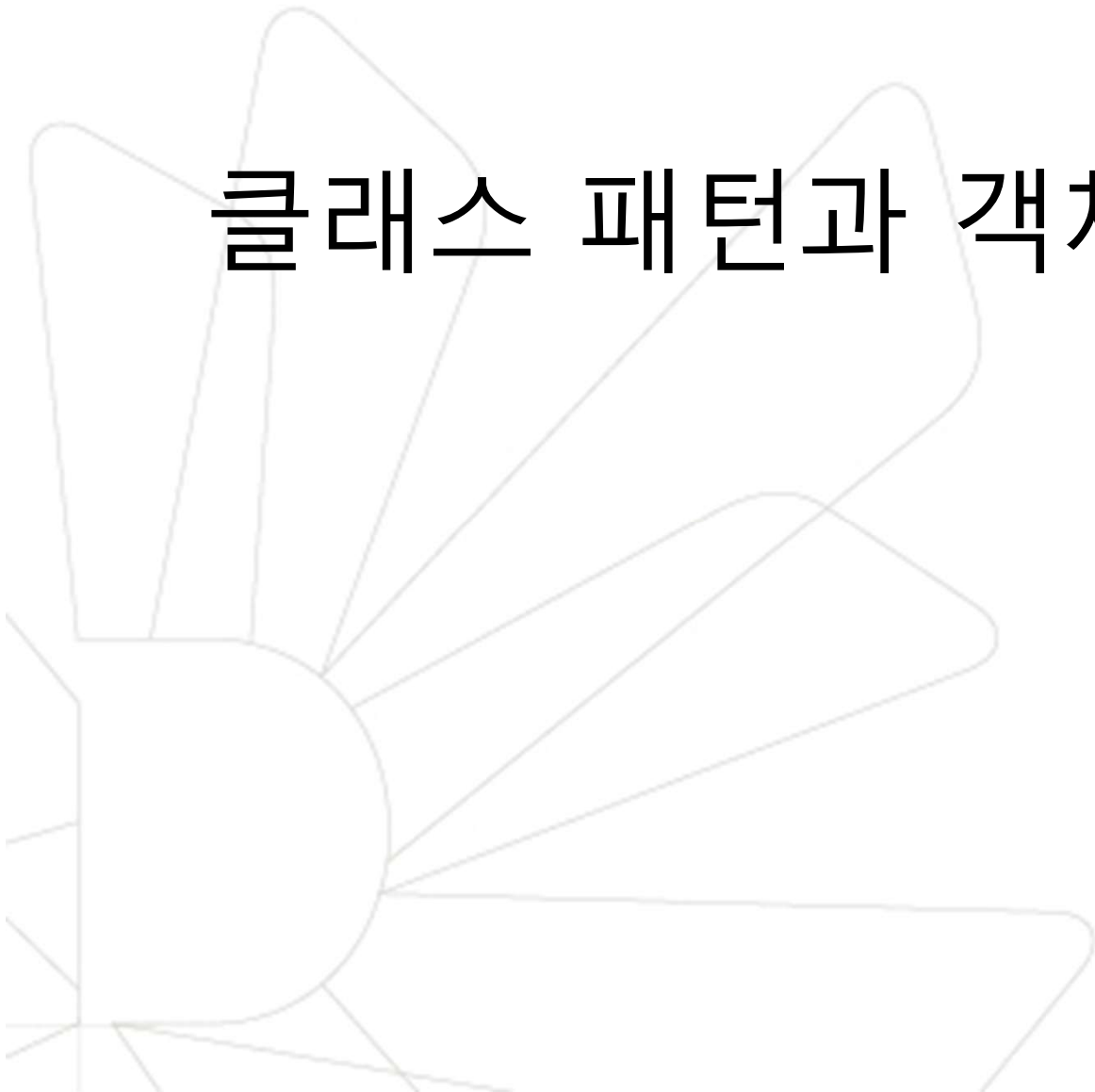


클래스 패턴과 객체 패턴의 차이



		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter(class) (139)	Interpreter(243) Template Method(325)
	Object	Abstract Factory(87) Builder(97) Prototype(117) Singleton(127)	Adapter(object) (139) Bridge(151) Composite(163) Decorator(175) Facade(185) Flyweight(195) Proxy(207)	Chain of Responsibility(223) Command(233) Iterator(257) Mediator(273) Memento(283) Observer(293) State(305) Strategy(315) Visitor(331)

클래스 패턴과 객체 패턴의 차이

- 클래스 패턴
 - 상속을 통해 클래스들의 관계를 부모-자식 관계로 정의한다.
 - 클래스 상호의 관계가 정적으로 결정되는 패턴
- 객체 패턴
 - 런타임(Run-time; 실행시간)에 객체간의 관계 또는 관련성이 변경될 수 있는 동적인 패턴

클래스 패턴과 객체 패턴의 적용 사례

- 클래스 패턴
 - 추상 팩토리 패턴, 어댑터 패턴, 일부이지만 브리지 패턴
 - 상속을 통해 확장가능한 구조를 만들어야 할 때 클래스 범위로 접근한다.
- 객체 패턴
 - 객체들이 서로 협력하여 책임을 분산하고 동적으로 교체하거나 확장할 필요가 있을 때 객체 범위로 접근한다.



위임(Delegation)

객체 지향 프로그래밍에서 위임(Delegation)
의 장점

단순히 설명하면

- 클래스의 구체적인 메소드(함수, API)를 몰라도 “이것 좀 해줘!” 라고 말기면 변경/수정에 유연하고 클라이언트 설계와 구현이 수월해진다.
 - 다른 클래스의 구조를 다 알지 못해도 되고 그 클래스들의 변경/수정에 민감하지 않아도 된다.
- 특정 객체(주로 클라이언트)가 특정 기능을 직접 처리하지 않고 해당 기능을 가진 다른 객체에게 처리해달라고 제어를 넘겨주는 방법
 - 제어를 넘겨주는 방법 ➔ 호출

보다 형식적인 위임의 장점

1. 유연한 구성(composition over Inheritance)
 - 상속은 컴파일 시간에 결정되지만 위임은 실행시간에 객체가 교체되거나 선택할 수 있다.
2. 낮은 결합도(loosely coupling)
 - 클래스간의 강한 종속성을 낮춘다.
내부 구현이 바뀌어도 객체(클라이언트)
3. 다중 상속의 문제 해결 (상속의 부작용 방지)
 - 불필요한 부모 클래스의 메서드까지 물려받아야 하는 '폭발적 상속'이나 '깨지기 쉬운 기반 클래스' 문제를 방지할 수 있다.
4. 코드의 재사용성의 향상과 모듈화
 - 필요한 기능을 가진 객체를 포함(Has-A)해서 사용하기 때문에 복잡한 기능을 독립적인 모듈 단위로 나누어져서 재사용이 쉽다.
5. SRP(Single Responsibility Principle)
 - 기능적인 부분을 외부의 전문적인 클래스가 수행하기 때문에 클래스에는 해당 위임받을 객체를 호출하는 정도만 있게 된다.